



# **Créer et intégrer une fonction Upsell à un e- commerce existant**

# Table des matières

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>I - Création de la fonction Upsell IA</b>        | <b>3</b>  |
| <b>II - Aller plus loin avec la fonction Upsell</b> | <b>22</b> |
| <b>III - Synthèse</b>                               | <b>25</b> |
| <b>IV - Auto-évaluation</b>                         | <b>27</b> |
| A. Exercice : Quiz .....                            | 27        |
| B. Exercice : Cas pratique.....                     | 31        |
| <b>Solutions des exercices</b>                      | <b>32</b> |

# I Création de la fonction Upsell IA

En marketing, un upsell consiste à proposer au client un produit complémentaire ou une version supérieure de ce qu'il a initialement choisi.

Dans le contexte d'un e-commerce, l'IA peut suggérer automatiquement des produits complémentaires à ajouter au panier afin d'augmenter le panier moyen.

## Qu'est-ce que le panier moyen ?

Az Définition

Le panier moyen est un indicateur qui mesure la dépense moyenne d'un client lors d'une commande en ligne ou en magasin.

Pour le calculer il suffit de diviser le chiffre d'affaires de l'entreprise par le nombre de commandes.

Ainsi, si un site e-commerce fait 10 000 € de ventes et qu'il a obtenu 200 commandes, le panier moyen est de 50 €.

Cela signifie qu'en moyenne chaque client dépense 50 € par commande sur le site.

Ainsi, chaque suggestion, acceptée peut à elle seule générer des dizaines de milliers d'euros supplémentaires par an à l'entreprise. Mais pour atteindre ce résultat, l'IA devra être pertinente et facile à tester. Il doit également être possible d'apporter facilement des améliorations sur le long terme, sans avoir à réécrire tout le code.

Notre version 1 de l'upsell permettra de suggérer automatiquement un produit à ajouter au panier, en fonction des articles déjà sélectionnés par l'utilisateur.

Pour commencer, nous allons créer un prompt.

## Pourquoi créer un prompt versionné ?

Remarque

En IA, la façon dont on pose la question change toute la réponse. Versionner chaque prompt c'est être certain de garder la main sur la qualité, tester différentes formulations et revenir en arrière si besoin est. Ce qu'on veut éviter c'est coder le prompt en dur car cela rend le testing plus compliqué.

Rendez-vous dans votre backend/core/prompts.

```
demo@dev cd backend
```

```
demo@dev cd core
```

```
demo@dev cd prompts
```

```
demo@dev touch upsell_prompt_v1.0.txt
```

```
1 cd backend
2 cd core
3 cd prompts
4 touch upsell_prompt_v1.0.txt
```

Le prompt sera en anglais car les modèles IA open source comprennent mieux l'anglais. Cela n'est pas dérangerant pour notre V1 mais il sera possible dans un second temps de tester d'autres modèles IA et donc également des prompts en français.

```
backend > core > prompts > if upsell_prompt_v1.0.txt
1 You are an assistant IA. A customer has these items in the cart: {cart_items}. Suggest one complementary product and briefly explain why.
```

```
1 You are an assistant IA. A customer has these items in the cart: {cart_items}.
  Suggest one complementary product and briefly explain why.
```

Traduction : Tu es un assistant IA. Un client a ces articles dans son panier : {cart\_items}. Suggère-lui un produit complémentaire et explique brièvement pourquoi.

Ensuite, nous allons nous rendre dans core/prompt\_manager afin de versionner nos prompts.

```
demo@dev cd ..
```

```
demo@dev cd core
```

```
1 cd ..
2 cd core
```

### Pourquoi centraliser le chargement des prompts ?

Remarque

Nous centralisons le chargement des prompts afin d'éviter la répétition et garantir que chaque fonction IA va charger le bon prompt versionné. On crée ainsi une fonction générique.

De ce fait, en fonction des tests que nous allons effectuer, il nous suffira de créer un autre fichier comme par exemple « *upsell\_prompt\_v1.1.txt* », puis de charger dynamiquement notre prompts dans notre fichier services.py

### Créer la logique métier pour la centralisation des prompts

Méthode

Commençons tout d'abord par créer la logique métier qui nous permettra de centraliser nos prompts.

1. Dans le fichier prompt\_manager.py, nous allons commencer par importer le module os. Il nous permettra d'interagir avec nos différents fichiers.

```
import os # on importe le module os qui nous permet d'interagir avec le système de fichiers
```

```
1 import os # on importe le module os qui nous permet d'interagir avec le système
  de fichiers
```

2. Ensuite, nous allons commencer par créer la fonction "load\_prompt".

Celle-ci va prendre en argument "prompt\_filename". Cela fait référence au nom du fichier prompt à charger (ici "upsell\_prompt\_v1.0.txt").

L'annotation "-> str" indique qu'on attend une chaîne de caractères et que l'on retournera aussi une chaîne de caractères.

```
def load_prompt(prompt_filename: str) -> str:
```

3. Nous construisons ensuite notre chemin absolu.

"\_\_file\_\_" permet de faire référence au chemin du script Python en cours.

Os.path.dirname(\_\_file\_\_) est le dossier où se trouve ce script

Os.path.join(...) assemble ce dossier avec le sous-dossier 'prompts' et le nom du fichier donné en argument.

```
prompt_path = os.path.join(os.path.dirname(__file__), 'prompts', prompt_filename)
```

4. Enfin, on ouvre le fichier en lecture 'r' avec l'encodage UTF-8

Le "with" garantit que le fichier sera refermé automatiquement après l'usage

F est l'objet fichier.

Puis on lit tout le contenu du fichier, et on le retourne sous forme de chaîne de caractères.

```
with open(prompt_path, 'r', encoding='utf-8') as f:
    return f.read()
```

Résultat ? Quand on appelle « *load\_prompt* », on obtient dans notre variable tout le texte du prompt prêt à être utilisé par notre IA.

Le résultat final est le suivant :

```
backend > core > prompt_manager.py > ...
1 import os # on importe le module os qui nous permet d'interagir avec le système de fichiers
2
3 def load_prompt(prompt_filename: str) -> str:
4     prompt_path = os.path.join(os.path.dirname(__file__), 'prompts', prompt_filename)
5     with open(prompt_path, 'r', encoding='utf-8') as f:
6         return f.read()
7
```

```
1 import os # on importe le module os qui nous permet d'interagir avec le système de
  fichiers
2 def load_prompt(prompt_filename: str) -> str:
3     prompt_path = os.path.join(os.path.dirname(__file__), 'prompts', prompt_filename)
4     with open(prompt_path, 'r', encoding='utf-8') as f:
5         return f.read()
```

Ceci étant fait, nous pouvons à nouveau nous rendre dans services.py pour y importer notre prompt manager, notre llm (Large Language Model) et inscrire notre logique métier.

#### Méthode

On commence par importer notre prompt et notre modèle IA.

```
from core.prompt_manager import load_prompt
from llama_cpp import Llama
```

```
1 from core.prompt_manager import load_prompt
2 from llama_cpp import Llama
```

Puis on crée notre constante PROMPT\_UPSELL. Elle contient le nom du fichier de notre prompt.

```
PROMPT_UPSELL = 'upsell_prompt_v1.0.txt'
```

```
1 PROMPT_UPSELL = 'upsell_prompt_v1.0.txt'
```

Puis nous allons définir la variable `MODEL_PATH`. Celle-ci indique où se trouve notre modèle IA Mistral (.gguf), dans le dossier `models`.

```
MODEL_PATH = os.path.join(
    os.path.dirname(__file__),
    '..',
    'models',
    'mistral-7b-q4_k_m.gguf'
```

```
1 MODEL_PATH = os.path.join(
2 os.path.dirname(__file__),
3 '..',
4 'models',
5 'mistral-7b-q4_k_m.gguf'
```

Maintenant, nous allons charger notre modèle IA avec `llama` en précisant plusieurs éléments :

Tout d'abord, le chemin du modèle (`model_path`)

Ensuite, combien de couches exécuter sur le GPU (`n_gpu_layers`)

Et enfin, combien de threads CPU utiliser (`n_threads`)

Pour le premier test, nous vous recommandons de laisser les paramètres tels quels.

```
llm = Llama(model_path=MODEL_PATH, n_gpu_layers=16, n_threads=4)
```

```
1 llm = Llama(model_path=MODEL_PATH, n_gpu_layers=16, n_threads=4)
```

Ensuite, nous allons définir notre fonction `get_upsell`. Celle-ci va prendre en paramètre `cart_items`. Qui est une liste de chaînes de caractères (ici les articles du client).

La fonction va retourner un dictionnaire (dict)

```
def get_upsell(cart_items: list[str]) -> dict:
```

```
1 def get_upsell(cart_items: list[str]) -> dict:
```

Ensuite nous allons utiliser notre fonction utilitaire `load_prompt`. Son rôle est de charger notre prompt (`upsell_prompt_v1.0.txt`.)

```
prompt_template = load_prompt(PROMPT_UPSELL)
```

```
1 prompt_template = load_prompt(PROMPT_UPSELL)
```

Ensuite, nous allons récupérer le texte du prompt, et remplacer `{cart_items}` par la liste d'articles du panier, jointes avec des virgules.

```
prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
```

```
1 prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
```

On envoie ensuite le prompt final au llm en lui demandant une réponse de maximum 50 tokens.

```
result = llm(prompt, max_tokens=50)
```

```
1 result = llm(prompt, max_tokens=50)
```

Puis enfin, on retourne un dictionnaire avec une clé « *suggestion* ». Cette valeur correspond à un texte généré par le LLM que l'on récupère (["choices"][0]["text"]) puis qu'on nettoie avec .strip()

```
return {"suggestion": result["choices"][0]["text"].strip()}
```

Le résultat final :

```
def get_upsell(cart_items: list[str]) -> dict:
    prompt_template = load_prompt(PROMPT_UPSELL)
    prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
    result = llm(prompt, max_tokens=50)
    return {"suggestion": result["choices"][0]["text"].strip()}
```

```
1 def get_upsell(cart_items: list[str]) -> dict:
2     prompt_template = load_prompt(PROMPT_UPSELL)
3     prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
4     result = llm(prompt, max_tokens=50)
5     return {"suggestion": result["choices"][0]["text"].strip()}
```

Et le fichier services.py au complet.

```

backend > core > services.py > ...
1 import json
2 import os
3 import torch
4 from transformers import pipeline
5
6 from core.prompt_manager import load_prompt
7 from llama_cpp import Llama
8
9 PROMPT_UPSELL = 'upsell_prompt_v1.0.txt'
10
11 MODEL_PATH = os.path.join(
12     os.path.dirname(__file__),
13     '..',
14     'models',
15     'mistral-7b-q4_k_m.gguf'
16 )
17
18 llm = Llama(model_path=MODEL_PATH, n_gpu_layers=16, n_threads=4)
19
20
21 USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
22 USE_DEMO_SENTIMENT = os.environ.get("USE_DEMO_SENTIMENT", "0") == "1"
23 _sentiment_pipeline = None # Variable pour lazy loading
24
25 DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")
26
27 def load_products():
28     with open(DATASET_PATH, 'r', encoding='utf-8') as f:
29         return json.load(f)
30
31 PRODUCTS_DATA = load_products()
32
33 def find_product_by_name(query: str):
34     query = query.lower()
35     for product in PRODUCTS_DATA:
36         if product['name'].lower() in query:
37             return product
38     return None
39
40 def suggest_wine_for_product(product):
41     pairing = {
42         "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
43         "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
44         "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
45         "Dessert": "Un vin moelleux, type Sauternes"
46     }
47     suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon vos goûts !")
48     return f"Pour {product['name']}, je te recommande {suggestion}"
49
50 def get_product_info(user_message: str):
51     if USE_DEMO_TEXT:
52         return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
53     product = find_product_by_name(user_message)
54     if not product:
55         return None
56
57     message_lower = user_message.lower()
58     if any(x in message_lower for x in [
59         "que contient",
60         "description",
61         "qu'y a-t-il dans",
62         "c'est quoi",
63         "qu'est-ce que"
64     ]):
65         return f"{product['name']} : {product['description']}"
66     else:
67         return suggest_wine_for_product(product)
68
69 # Analyse de sentiment
70 def get_sentiment_pipeline():
71     """
72     Charge le pipeline HuggingFace pour l'analyse de sentiment.
73     Utilise PyTorch en backend.
74     Chargement unique (singleton).
75     """
76     global _sentiment_pipeline
77     if _sentiment_pipeline is None:
78         _sentiment_pipeline = pipeline("sentiment-analysis")
79     return _sentiment_pipeline
80
81 def analyze_sentiment(text):
82     if USE_DEMO_SENTIMENT:
83         return "neutral", 0.5
84
85     pipe = get_sentiment_pipeline()
86     result = pipe(text)
87     label = result[0]["label"].lower()
88     score = result[0]["score"]
89
90     return label, score
91
92 # ---- Fonction upsell
93 def get_upsell(cart_items: list[str]) -> dict:
94     prompt_template = load_prompt(PROMPT_UPSELL)
95     prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
96     result = llm(prompt, max_tokens=50)
97     return {"suggestion": result["choices"][0]["text"].strip()}
98

```

```

1 import json
2 import os
3 import torch
4 from transformers import pipeline
5
6 from core.prompt_manager import load_prompt

```

```

7 from llama_cpp import Llama
8
9 PROMPT_UPSELL = 'upsell_prompt_v1.0.txt'
10
11 MODEL_PATH = os.path.join(
12 os.path.dirname(__file__),
13 '..',
14 'models',
15 'mistral-7b-q4_k_m.gguf'
16 )
17
18 llm = Llama(model_path=MODEL_PATH, n_gpu_layers=16, n_threads=4)
19
20
21 USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
22 USE_DEMO_SENTIMENT = os.environ.get("USE_DEMO_SENTIMENT", "0") == "1"
23 _sentiment_pipeline = None # Variable pour lazy loading
24
25 DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")
26
27 def load_products():
28 with open(DATASET_PATH, 'r', encoding='utf-8') as f:
29 return json.load(f)
30
31 PRODUCTS_DATA = load_products()
32
33 def find_product_by_name(query: str):
34 query = query.lower()
35 for product in PRODUCTS_DATA:
36 if product['name'].lower() in query:
37 return product
38 return None
39
40 def suggest_wine_for_product(product):
41 pairing = {
42 "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
43 "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
44 "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
45 "Dessert": "Un vin moelleux, type Sauternes"
46 }
47 suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon
    vos goûts !")
48 return f"Pour {product['name']}, je te recommande {suggestion}"
49
50 def get_product_info(user_message: str):
51 if USE_DEMO_TEXT:
52 return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
53 product = find_product_by_name(user_message)
54 if not product:
55 return None
56
57 message_lower = user_message.lower()
58 if any(x in message_lower for x in [
59 "que contient",
60 "description",
61 "qu'y a-t-il dans",
62 "c'est quoi",
63 "qu'est-ce que"
64 ]):
65 return f"{product['name']} : {product['description']}"
66 else:
67 return suggest_wine_for_product(product)
68

```

```

69 # Analyse de sentiment
70 def get_sentiment_pipeline():
71     """
72     Charge le pipeline HuggingFace pour l'analyse de sentiment.
73     Utilise PyTorch en backend.
74     Chargement unique (singleton).
75     """
76     global _sentiment_pipeline
77     if _sentiment_pipeline is None:
78         _sentiment_pipeline = pipeline("sentiment-analysis")
79     return _sentiment_pipeline
80
81 def analyze_sentiment(text):
82     if USE_DEMO_SENTIMENT:
83         return "neutral", 0.5
84
85     pipe = get_sentiment_pipeline()
86     result = pipe(text)
87     label = result[0]["label"].lower()
88     score = result[0]["score"]
89
90     return label, score
91
92 # ---- Fonction upsell
93
94
95 def get_upsell(cart_items: list[str]) -> dict:
96     prompt_template = load_prompt(PROMPT_UPSELL)
97     prompt = prompt_template.replace("{cart_items}", ", ".join(cart_items))
98     result = llm(prompt, max_tokens=50)
99     return {"suggestion": result["choices"][0]["text"].strip()}

```

## Création de l'endpoint Upsell

 Méthode

Ensuite, nous allons construire notre endpoint exactement comme pour nos précédents endpoint.

1. Nous allons commencer par importer notre fonction `get_upsell`

```
from core.services import get_upsell
```

```
1 from core.services import get_upsell
```

2. Ensuite, nous allons créer notre route FastAPI en fin de fichier `main.py` juste avant notre middleware CORS.

Grâce à cet endpoint, lorsqu'un client va envoyer une requête POST sur l'URL `/upsell`, notre fonction sera appelée.

Nous allons également préciser que la réponse doit respecter notre modèle Pydantic `UpsellResponse` (c'est une classe que nous allons définir dans un second temps).

```
@app.post("/upsell", response_model=UpsellResponse)
```

```
1 @app.post("/upsell", response_model=UpsellResponse)
```

3. Ensuite, nous allons définir la fonction `upsell`. Celle-ci sera exécutée quand la route sera appelée.

Elle est asynchrone (async def) et le paramètre request: UpsellRequest signifie que le corps de la requête POST doit correspondre au modèle Pydantic UpsellRequest que nous allons également créer dans un second temps.

```
async def upsell(request: UpsellRequest):
```

```
1 async def upsell(request: UpsellRequest):
```

4. Désormais, nous allons appliquer notre fonction get\_upsell() en lui passant la liste des articles du panier (request.cart\_items).

```
result = get_upsell(request.cart_items)
```

```
1 result = get_upsell(request.cart_items)
```

5. Enfin, nous allons retourner la réponse.

```
return UpsellResponse(**result)
```

```
1 return UpsellResponse(**result)
```

Résultat final attendu :

```
1 @app.post("/upsell", response_model=UpsellResponse)
2 async def upsell(request: UpsellRequest):
3     result = get_upsell(request.cart_items)
4     return UpsellResponse(**result)
```

6. Nous allons importer nos schémas pydantic en début de fichier main.py, aux côtés des schémas Pydantic liés à notre endpoint /chat.

```
from api.schemas import ChatRequest, ChatResponse, UpsellRequest, UpsellResponse
```

```
1 from api.schemas import ChatRequest, ChatResponse, UpsellRequest, UpsellResponse
```

Dorénavant, nous pouvons définir nos schémas pydantic. Ces schémas pydantic nous permettront de garantir que chaque appel frontend/backend contienne bien la bonne structure.

```
demo@dev cd schemas.py
```

```
# On définit les schémas de requête
class UpsellRequest(BaseModel):
    |   cart_items: List[str]
```

```
# On définit les schémas de réponse
class UpsellResponse(BaseModel):
    |   suggestion: str
```

```
1 cd schemas.py
```

```

1 # On définit les schémas de requête
2 class UpsellRequest(BaseModel):
3     cart_items: List[str]
4 # On définit les schémas de réponse
5 class UpsellResponse(BaseModel):
6     suggestion: str

```

Pour un résultat final :

```

backend > api > 📄 schemas.py > ...
1     from pydantic import BaseModel
2     from typing import List, Optional
3
4
5     class ChatRequest(BaseModel):
6         session_id: Optional[str] = None
7         user_message: str
8         products: Optional[List[str]] = []
9
10    class ChatResponse(BaseModel):
11        recommendation: str
12        sentiment: Optional[str] = None
13        escalate_to_human: bool = False
14
15    # On définit les schémas de requête
16    class UpsellRequest(BaseModel):
17        cart_items: List[str]
18
19    # On définit les schémas de réponse
20    class UpsellResponse(BaseModel):
21        suggestion: str
22

```

```

1 from pydantic import BaseModel
2 from typing import List, Optional
3
4 class ChatRequest(BaseModel):
5     session_id: Optional[str] = None
6     user_message: str
7     products: Optional[List[str]] = []
8
9 class ChatResponse(BaseModel):
10     recommendation: str
11     sentiment: Optional[str] = None
12     escalate_to_human: bool = False
13
14 # On définit les schémas de requête
15 class UpsellRequest(BaseModel):
16     cart_items: List[str]

```

```
17
18 # On définit les schémas de réponse
19 class UpsellResponse(BaseModel):
20     suggestion: str
```

Il est temps de réaliser nos tests.

Pour ce faire, nous allons tout d'abord nous rendre dans le frontend et ajouter une route à notre application.

### Ajouter une route à notre application

[Méthode](#)

La première étape consiste à se rendre dans `src/App.jsx`, importer notre page Upsell, notre composant upsell et ajouter la route upsell.

```
demo@dev cd frontend
```

```
demo@dev cd src
```

```
import UpsellPage from "../pages/UpsellPage";
import Upsell from "../components/Upsell";
```

```
1 cd frontend
2 cd src
```

```
1 import UpsellPage from "../pages/UpsellPage";
2 import Upsell from "../components/Upsell";
```

```
<Route path="/upsell" element={<UpsellPage />} />
```

```
1 <Route path="/upsell" element={<UpsellPage />} />
```

Pour un résultat final :

```

frontend > src > App.jsx > ...
1  import { BrowserRouter, Routes, Route } from "react-router-dom";
2  import ChatPage from "../pages/ChatPage";
3  import Navbar from "../components/Navbar";
4  import UpsellPage from "../pages/UpsellPage";
5  import Upsell from "../components/Upsell";
6
7  export default function App() {
8    return (
9      <BrowserRouter>
10       <div className="min-h-screen bg-white">
11         {/* NAVBAR importée */}
12         <Navbar />
13
14         <main className="flex justify-center items-start py-10">
15           <Routes>
16             <Route path="/" element={<ChatPage />} />
17             <Route path="/upsell" element={<UpsellPage />} />
18           </Routes>
19         </main>
20       </div>
21     </BrowserRouter>
22   );
23 }

```

```

1 import { BrowserRouter, Routes, Route } from "react-router-dom";
2 import ChatPage from "../pages/ChatPage";
3 import Navbar from "../components/Navbar";
4 import UpsellPage from "../pages/UpsellPage";
5 import Upsell from "../components/Upsell";
6
7 export default function App() {
8   return (
9     <BrowserRouter>
10      <div className="min-h-screen bg-white">
11        {/* NAVBAR importée */}
12        <Navbar />
13
14        <main className="flex justify-center items-start py-10">
15          <Routes>
16            <Route path="/" element={<ChatPage />} />
17            <Route path="/upsell" element={<UpsellPage />} />
18          </Routes>
19        </main>
20      </div>
21    </BrowserRouter>
22  );
23 }

```

Ensuite, nous allons nous rendre dans pages/UpsellPage et créer le container de notre page. Nous allons également importer notre composant Upsell.

```
demo@dev cd pages
```

```
frontend > src > pages > UpsellPage.jsx > ...  
1 import Upsell from "../components/Upsell";  
2  
3 export default function UpsellPage() {  
4   return (  
5     <div className="w-full max-w-xl">  
6       <h1 className="text-3xl font-bold mb-8 text-center">  
7         Démo Upsell e-commerce  
8       </h1>  
9       <Upsell />  
10    </div>  
11  );  
12 }  
13
```

```
1 import Upsell from "../components/Upsell";  
2  
3 export default function UpsellPage() {  
4   return (  
5     <div className="w-full max-w-xl">  
6     <h1 className="text-3xl font-bold mb-8 text-center">  
7     Démo Upsell e-commerce  
8     </h1>  
9     <Upsell />  
10    </div>  
11  );  
12 }
```

```
demo@dev cd ..
```

```
demo@dev cd components
```

Enfin, nous allons nous rendre dans le dossier src/components pour modifier le fichier Upsell.jsx  
C'est ce fichier qui permettra d'afficher toute la logique de notre fonction Upsell.

[illegible]

```
1 import React, { useState } from "react";
2
3 // Mini-liste pour la démo
4 const PRODUCTS = [
5   { label: "Brunch Estival", emoji: "🥞" },
6   { label: "Jus de Raisin", emoji: "🍇" },
7   { label: "Pinot Noir", emoji: "🍷" },
8   { label: "Brunch Saint Valentin", emoji: "💕" },
9 ]
```

```

9 { label: "Fromage de Chèvre", emoji: "🧀" },
10 { label: "Bière sans alcool", emoji: "🍺" },
11 ];
12
13 export default function Upsell() {
14   const [cart, setCart] = useState([]);
15   const [input, setInput] = useState("");
16   const [loading, setLoading] = useState(false);
17   const [suggestion, setSuggestion] = useState("");
18   const [error, setError] = useState("");
19
20   function handleQuickAdd(item) {
21     if (!cart.includes(item)) {
22       setCart([...cart, item]);
23       setSuggestion("");
24     }
25   }
26
27   function handleManualAdd(e) {
28     e.preventDefault();
29     if (input.trim() && !cart.includes(input.trim())) {
30       setCart([...cart, input.trim()]);
31       setInput("");
32       setSuggestion("");
33     }
34   }
35
36   function handleRemove(item) {
37     setCart(cart.filter((c) => c !== item));
38     setSuggestion("");
39   }
40
41   async function handleUpsell() {
42     setError("");
43     setSuggestion("");
44     setLoading(true);
45     try {
46       const response = await fetch("http://localhost:8000/upsell", {
47         method: "POST",
48         headers: { "Content-Type": "application/json" },
49         body: JSON.stringify({ cart_items: cart }),
50       });
51       if (!response.ok) throw new Error("Erreur serveur API");
52       const data = await response.json();
53       setSuggestion(data.suggestion);
54     } catch {
55       setError("Une erreur est survenue.");
56     }
57     setLoading(false);
58   }
59
60   function handleAddSuggestion() {
61     if (suggestion && !cart.includes(suggestion)) {
62       setCart([...cart, suggestion]);
63       setSuggestion("");
64     }
65   }
66
67   return (
68     <div className="max-w-lg mx-auto p-6 bg-white rounded-2xl shadow-lg mt-8">
69       <h2 className="text-2xl font-bold mb-4">Votre panier</h2>
70     </div>
71     /* Panier : tags modernisés */

```

```

72 <div className="flex flex-wrap gap-2 mb-4 min-h-[40px]">
73 {cart.length === 0 && (
74 <span className="text-gray-400 italic">Panier vide</span>
75 )}
76 {cart.map((item) => (
77 <span
78 key={item}
79 className="flex items-center gap-1 px-4 py-2 rounded-2xl bg-white text-gray-800
80   border border-gray-200 shadow-sm font-medium text-lg"
81 <span className="text-xl">{PRODUCTS.find((p) => p.label === item)?.emoji || "🛒"}
82 </span>
83 <button
84   className="ml-2 text-gray-400 hover:text-red-600 text-xl font-bold transition
85   focus:outline-none focus:ring-2 focus:ring-red-200"
86   onClick={() => handleRemove(item)}
87   title="Retirer"
88   type="button"
89   style={{
90     background: "transparent",
91     border: "none",
92     padding: 0,
93     lineHeight: 1,
94     cursor: "pointer",
95   }}
96 >
97 </button>
98 </span>
99 )})}
100 </div>
101
102 {/* Grille produits rapides */}
103 <div className="mb-4">
104 <div className="flex flex-wrap gap-2">
105 {PRODUCTS.map((p) => (
106 <button
107   key={p.label}
108   className={`border px-3 py-2 rounded-xl flex items-center gap-2 transition
109     ${
110       cart.includes(p.label)
111       ? "bg-green-50 border-green-300 text-green-800"
112       : "hover:bg-neutral-100 border-gray-200 bg-white text-gray-700"
113     }`}
114   onClick={() => handleQuickAdd(p.label)}
115   disabled={cart.includes(p.label)}
116 >
117 <span className="text-xl">{p.emoji}</span>
118 <span>{p.label}</span>
119 </button>
120 )})}
121 </div>
122 </div>
123
124 {/* Ajout manuel */}
125 <form onSubmit={handleManualAdd} className="flex gap-2 mb-4">
126 <input
127   className="flex-1 border rounded p-2"
128   placeholder="Ajouter autre produit..."
129   value={input}
130   onChange={(e) => setInput(e.target.value)}
131 />

```

```

132 <button
133 type="submit"
134 className="bg-gray-100 px-4 rounded font-semibold"
135 >
136 Ajouter
137 </button>
138 </form>
139
140 {/* Bouton upsell IA */}
141 <button
142 onClick={handleUpsell}
143 className="bg-blue-600 text-white rounded p-2 font-semibold hover:bg-blue-700 w-full
144 mb-2"
145 disabled={loading || cart.length === 0}
146 >
147 {loading ? "Recherche IA..." : "Suggérer un produit complémentaire"}
148 </button>
149 {/* Affichage suggestion */}
150 {suggestion && (
151 <div className="mt-4 p-3 rounded bg-green-50 border border-green-200 text-green-900
152 flex items-center justify-between">
153 <span>
154 <b>Suggestion IA :</b> {suggestion}
155 </span>
156 <button
157 onClick={handleAddSuggestion}
158 className="ml-4 px-3 py-1 rounded bg-green-600 text-white font-semibold hover:bg-
159 green-700"
160 >
161 Ajouter au panier
162 </button>
163 </div>
164 )}
165 {/* Erreur */}
166 {error && (
167 <div className="mt-4 p-3 rounded bg-red-50 border border-red-200 text-red-900">
168 {error}
169 </div>
170 )}
171 </div>
172 }

```

Une fois réalisé, rendez-vous dans le backend, activez votre .venv si ce n'est pas déjà fait et relancez votre serveur côté backend.

Faites de même coté frontend.

```
demo@dev cd backend
```

```
demo@dev source .venv/bin/activate
```

```
demo@dev uvicorn api.main:app --reload
```

```
demo@dev cd frontend
```

```
demo@dev npm run dev
```

```
1 cd backend  
2 source .venv/bin/activate  
3 uvicorn api.main:app --reload  
4 cd frontend  
5 npm run dev
```

 Rappel

Voici les commandes de base pour votre serveur Uvicorn ainsi que le frontend.

- CTRL + C : permet d'arrêter un processus en cours
- source .venv/bin/activate permet d'activer votre environnement virtuel
- deactivate permet de désactiver votre environnement virtuel
- uvicorn api.main:app --reload : permet de lancer votre serveur
- Depuis le frontend, npm run dev permet de lancer votre application

Lorsque vous testez la fonction upsell, cela devrait afficher une recommandation personnalisée.

## Votre panier



Brunch Estival x



Fromage de Chèvre x



Brunch Estival



Jus de Raisin



Pinot Noir



Brunch Saint Valentin



Fromage de Chèvre



Bière sans alcool

Ajouter autre produit...

Ajouter

Suggérer un produit complémentaire

Suggestion IA : Based on the given items, a complementary product that would go well with Brunch Estival and Fromage de Chèvre is a bottle of Champagne or Sparkling Wine. The reason for this recommendation is that Champagne

Ajouter  
au  
panier

## II Aller plus loin avec la fonction Upsell

Imaginons la situation suivante.

Emilie réunit l'équipe lors d'une réunion afin de présenter la nouvelle fonction d'upsell IA. L'enthousiasme est tel que l'équipe souhaite déployer la solution dès le lendemain. À sa place, auriez-vous accepté ?

Accepter sans préparation serait une erreur stratégique.

En effet, déployer la V1 telle quelle c'est faire tourner le LLM sur un serveur local. Cela suffit pour démo interne mais dès que plusieurs clients utiliseront l'upsell en même temps, nous risquons des ralentissements, des coupures ou des échecs de suggestion et pire encore, cela risque de dégrader l'expérience utilisateur à un moment sensible : l'étape du paiement.

### 👁 Exemple

L'utilisateur utilise l'IA pour obtenir une suggestion. Celle-ci lui plaît mais depuis qu'il a cliqué sur l'option « *obtenir une suggestion personnalisée* », le site e-commerce semble être plus lent. Il met plusieurs secondes à charger. L'écran se fige et le client commence à douter. Il se demande s'il a été piraté ou s'il va être facturé deux fois.

Inquiet, il ferme la page.

Bon à savoir :

Au-delà de 3 secondes de latence sur une page de paiement le taux d'abandon du panier explose et un client qui quitte un site web à ce moment précis revient rarement.

La solution ici serait de passer sur un serveur GPU (processeur graphique) dédié tel que OVH cloud (OVHcloud<sup>1</sup>) ou encore AWS par Amazon (aws<sup>2</sup>).

En effet, étant donné qu'un GPU est conçu pour traiter des calculs massifs en parallèle, c'est exactement ce dont une IA a besoin pour générer du texte rapidement.

Cela s'oppose aux CPU (processeurs standards) qui sont optimisés pour exécuter un petit nombre de tâches complexes les unes après les autres mais qui se retrouvent vite limités lorsqu'il faut gérer des milliers d'opération en simultané.

De cette manière, nous gagnons en puissance car les calculs nécessaires pour faire tourner un modèle comme Llama sont exécutés plus vite.

Nous gagnons également en stabilité car le serveur peut absorber une charge plus importante. En clair, plusieurs utilisateurs pourraient interagir avec l'IA en même temps sans que cela engendre des ralentissements importants.

Deuxièmement, il pourrait être intéressant de tester une API cloud.

Nous pourrions par exemple utiliser un service comme Open AI ou encore Mistral.

Dans ce cas, nous n'avons plus besoin de déployer ou d'héberger nous-mêmes un modèle IA lourd sur nos serveurs. Tout serait géré par le prestataire. Très concrètement, nous envoyons une requête et nous recevons une réponse déjà calculée par leur serveurs GPU.

Les avantages sont nombreux.

1. <https://www.ovhcloud.com/fr/>

2. [https://aws.amazon.com/fr/?nc2=h\\_lg](https://aws.amazon.com/fr/?nc2=h_lg)

Tout d'abord, il y a la rapidité et la simplicité de mise en place. Une intégration par API suffit pour pouvoir exploiter le service. Cela permet de tester et d'itérer rapidement.

Ensuite, ces services ont des serveurs optimisés et mis à jour en continu. Cela garanti que le temps de réponse soit court et que la qualité de génération soit élevée.

Enfin, la fiabilité. L'API gère automatiquement la montée en charge et permet donc d'assurer un service en continue sans latences.

Cependant, cette solution comporte aussi des limites.

Le coût est l'un d'eux. En effet, en faisant appel à une API cloud, chaque requête est facturée. Cela peut devenir plus ou moins significatif en fonction du volume d'utilisateurs.

Il y a également la dépendance au service IA. La disponibilité et les performances dépendent directement d'un service externe (exemple OpenAI). Une modification dans la tarification ou la politique d'utilisation pourrait grandement impacter l'entreprise.

Dernier point, la confidentialité. Certaines données sensibles comme le contenu du panier des clients, transitent par ce service cloud. Cela peut poser des questions de conformités selon le pays dans lequel le service est proposé.

En France, cela est géré par la CNIL (CNIL<sup>3</sup>).

Ce sont donc tous les points à prendre en considération si l'on souhaite utiliser un service cloud.

Enfin parmi les autres options il y a la possibilité de mettre en place une file d'attente (queue).

L'avantage est que cela évite de saturer le serveur. Plutôt que de traiter toutes les requêtes en même temps (ce qui justement, peut engendrer un crash de l'application) les appels sont placés dans une file et traités dans l'ordre.

#### Exemple

À ses débuts, openAI avait mis en place un système de queue pour chatGPT.

Cela garantit que chaque requête sera bien exécutée même si l'utilisateur doit attendre quelques secondes supplémentaires.

L'inconvénient principal de cette méthode est que cela augmente le temps de latence côté utilisateur. Si les demandes s'accumulent, certains utilisateurs pourraient attendre plus ou moins longtemps avant d'obtenir une réponse. Pour une requête simple au chatbot, cela pourrait être envisageable mais a une étape aussi critique que le paiement, cette attente peut au contraire, frustrer le client et baisser le taux de conversion.

### Qu'est-ce qu'un taux de conversion ?

#### Az Définition

Le taux de conversion est un indicateur qui permet de calculer le pourcentage de visiteurs qui réalisent l'action que l'on attend d'eux.

Cette action est définie par l'entreprise (inscription à une newsletter, achat sur le site e-commerce, etc.)

Le taux de conversion c'est le nombre de conversions divisé par le nombre total de visiteurs x 100.

Ainsi, si on a 1 000 visiteurs sur un site e-commerce, et que 50 d'entre eux achètent le produit, le taux de conversion est donc de  $50 \div 1\,000 \times 100 = 5\%$

<sup>3</sup>. <https://www.cnil.fr/fr>

Nous pourrions également limiter la taille des prompts ou la fréquence des appels pour éviter les abus mais cela risque de nuire à l'expérience utilisateur.

Finalement, qu'importe la solution choisie, le plus important est de ne jamais déployer une solution sans effectuer de tests au préalable. Une V2 doit être murement réfléchie et évaluée en prenant en compte les avantages comme les inconvénients.

## III Synthèse

### A. Flashcards

#### Upsell

Proposer un produit complémentaire pour augmenter la valeur du panier moyen.

#### Panier moyen

Dépense moyenne d'un client par commande sur un site e-commerce.

#### Prompt versionné

Question IA enregistrée en version pour faciliter tests et améliorations.

#### Centralisation des prompts

Gestion unique des prompts pour éviter répétitions et erreurs.

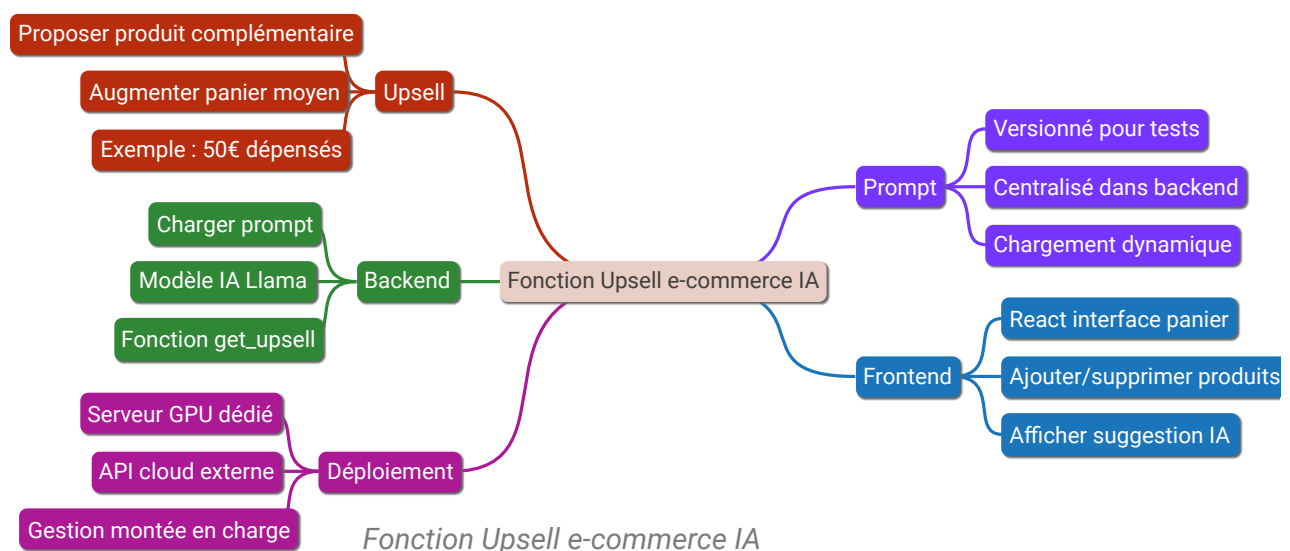
#### Serveur GPU dédié

Serveur optimisé pour calculs IA rapides et gestion de charge élevée.

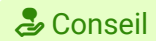
### La théorie en Bref

💡 Fondamental

L'**upsell** en e-commerce consiste à proposer un produit complémentaire ou supérieur pour augmenter le **panier moyen**, qui mesure la dépense moyenne par commande. La fonction Upsell IA utilise un **prompt versionné** pour suggérer automatiquement un produit pertinent selon les articles du panier, facilitant ainsi les tests et améliorations. La **centralisation des prompts** dans le backend permet une gestion efficace et évolutive des versions. L'intégration se fait via un **endpoint FastAPI** qui communique avec le frontend React, où l'utilisateur peut ajouter des produits et recevoir des suggestions. Pour une montée en charge, il est recommandé d'utiliser un serveur GPU dédié ou une API cloud, en tenant compte des coûts, confidentialité et performances.



## Les conseils pro



### Comment intégrer une fonction Upsell IA dans un e-commerce ?

- Créer un prompt versionné pour flexibilité
- Centraliser le chargement des prompts
- Utiliser un modèle IA adapté
- Développer un endpoint API fiable

### Comment garantir la pertinence des suggestions Upsell ?

- Analyser les articles du panier
- Tester différentes versions de prompt
- Impliquer les équipes marketing
- Collecter les retours utilisateurs

### Comment assurer la performance du service Upsell ?

- Utiliser un serveur GPU dédié
- Envisager une API cloud
- Mettre en place une file d'attente
- Limiter la taille des prompts

### Comment gérer la montée en charge du service Upsell ?

- Prévoir un hébergement scalable
- Surveiller la latence utilisateur
- Optimiser les appels IA
- Tester avant déploiement

### Comment protéger la confidentialité des données clients ?

- Évaluer la conformité CNIL
- Limiter les données envoyées
- Préférer solutions internes si sensible
- Informer les utilisateurs

## IV Auto-évaluation

### A. Exercice : Quiz

[solution n°1 p. 32]

#### Exercice

Quelle est la principale fonction de la fonction Upsell dans un e-commerce ?

Proposer un produit complémentaire pour augmenter le panier moyen

Réduire le nombre d'articles dans le panier

Calculer le taux de conversion

Analyser le sentiment des clients

#### Exercice

Le prompt versionné est créé pour faciliter les tests et permettre de revenir à une version antérieure si nécessaire. Vrai ou Faux ?

Vrai

Faux

#### Exercice

Pourquoi centraliser le chargement des prompts dans le backend ?

Pour éviter la répétition et garantir que chaque fonction IA charge le bon prompt versionné

Pour accélérer le temps de réponse du serveur

Pour réduire la taille des prompts

Pour limiter le nombre de requêtes API

#### Exercice

Quels sont les avantages d'utiliser un serveur GPU dédié pour l'IA Upsell ?

**A** Traitement plus rapide des calculs IA

**B** Meilleure gestion de la montée en charge

**C** Réduction des coûts d'hébergement

**D** Amélioration de la stabilité du service

### Exercice

Quels sont les avantages et inconvénients d'utiliser une API cloud pour la fonction Upsell ?

**A** Avantages : rapidité et simplicité d'intégration

**B** Avantages : gestion automatique de la montée en charge

**C** Inconvénients : coût par requête

**D** Inconvénients : dépendance à un service externe

**E** Inconvénients : confidentialité des données

### Exercice

Quel est l'inconvénient principal de mettre en place une file d'attente (queue) pour gérer les requêtes IA ?

Augmentation du temps de latence côté utilisateur

Risque de surcharge du serveur

Complexité de développement

Perte de données

### Exercice

Comment calcule-t-on le taux de conversion sur un site e-commerce ?

Nombre de conversions divisé par nombre total de visiteurs multiplié par 100

Nombre de visiteurs divisé par nombre de conversions multiplié par 100

Nombre de commandes divisé par chiffre d'affaires

Chiffre d'affaires divisé par nombre de commandes

### Exercice

Dans le frontend React, l'utilisateur peut ajouter des produits au panier, demander une suggestion IA et ajouter la suggestion au panier. Vrai ou Faux ?

Vrai

Faux

### Exercice

Quel est le rôle de l'endpoint /upsell dans le backend FastAPI ?

Recevoir la liste des articles du panier et retourner une suggestion de produit complémentaire

Calculer le panier moyen

Analyser le sentiment des messages clients

Gérer l'authentification des utilisateurs

### Exercice

Quelles recommandations sont pertinentes pour une version 2 (V2) de la fonction Upsell ?

**A** Utiliser un serveur GPU dédié pour la montée en charge

**B** Tester différentes versions de prompt

**C** Mettre en place une API cloud

**D** Limiter la taille des prompts

**E** Déployer directement sans tests

### Exercice

Associez les concepts à leur définition ou rôle

**A** Prompt versionné

**B** Panier moyen

**C** Upsell

**D** Serveur GPU dédié

|                                                                   |                              |                                     |                                                       |
|-------------------------------------------------------------------|------------------------------|-------------------------------------|-------------------------------------------------------|
| Proposer un produit complémentaire pour augmenter le panier moyen | Dépense moyenne par commande | Facilite les tests et améliorations | Optimisé pour calculs IA rapides et gestion de charge |
|-------------------------------------------------------------------|------------------------------|-------------------------------------|-------------------------------------------------------|

### Exercice

Placez dans l'ordre correct les étapes pour intégrer la fonction Upsell dans le backend.

**A** Centraliser le chargement des prompts

**B** Créer l'endpoint /upsell

**C** Définir la fonction get\_upsell

**D** Créer le prompt versionné

**E** Charger le modèle IA

Réponse : \_\_\_\_ \_\_\_\_ \_\_\_\_ \_\_\_\_ \_\_\_\_

### Exercice

Placez dans l'ordre les actions réalisées dans le frontend React pour utiliser la fonction Upsell.

**A** Envoyer la liste au backend via POST /upsell

**B** Afficher la suggestion

**C** Ajouter la suggestion au panier

**D** Ajouter des produits au panier

**E** Recevoir la suggestion IA

Réponse : \_\_\_\_ \_\_\_\_ \_\_\_\_ \_\_\_\_ \_\_\_\_

### Exercice

Associez les avantages et inconvénients à l'utilisation d'une API cloud pour l'Upsell

**A** Inconvénients

**B** Avantages

|                                                |                                   |
|------------------------------------------------|-----------------------------------|
| Rapidité, simplicité, gestion montée en charge | Coût, dépendance, confidentialité |
|------------------------------------------------|-----------------------------------|

### Exercice

Associez les caractéristiques au serveur GPU dédié ou à l'API cloud

**A** API cloud

**B** Serveur GPU dédié

|                                                      |                                                                      |
|------------------------------------------------------|----------------------------------------------------------------------|
| Exécution locale, nécessite gestion montée en charge | Service externe, facturation à l'usage, montée en charge automatique |
|------------------------------------------------------|----------------------------------------------------------------------|

## Exercice

Placez dans l'ordre les étapes recommandées avant de déployer la fonction Upsell en production.

**A** Choisir entre serveur GPU ou API cloud

**B** Mettre en place la gestion des erreurs et files d'attente

**C** Déployer progressivement

**D** Évaluer la montée en charge

**E** Tester la fonction en interne

Réponse : \_\_\_\_ \_

## B. Exercice : Cas pratique

### Question 1

[solution n°2 p. 37]

Décrivez le fonctionnement de la fonction Upsell côté front end et backend.

### Question 2

[solution n°3 p. 37]

Donnez également des recommandations en vue d'une v2 de la fonction.

# Solutions des exercices

## Solution n°1

[exercice p. 27]

### Exercice

Quelle est la principale fonction de la fonction Upsell dans un e-commerce ?

✓ Proposer un produit complémentaire pour augmenter le panier moyen

Réduire le nombre d'articles dans le panier

Calculer le taux de conversion

Analyser le sentiment des clients

🔍 La fonction Upsell vise à suggérer un produit complémentaire ou une version supérieure afin d'augmenter la dépense moyenne par commande, appelée panier moyen.

### Exercice

Le prompt versionné est créé pour faciliter les tests et permettre de revenir à une version antérieure si nécessaire. Vrai ou Faux ?

✓ Vrai

Faux

🔍 Versionner les prompts permet de tester différentes formulations et de garantir la qualité des suggestions sans avoir à réécrire tout le code.

### Exercice

Pourquoi centraliser le chargement des prompts dans le backend ?

✓ Pour éviter la répétition et garantir que chaque fonction IA charge le bon prompt versionné

Pour accélérer le temps de réponse du serveur

Pour réduire la taille des prompts

Pour limiter le nombre de requêtes API

🔍 La centralisation évite la duplication de code et assure que chaque fonction utilise la version correcte du prompt, facilitant ainsi la maintenance et les mises à jour.

### Exercice

Quels sont les avantages d'utiliser un serveur GPU dédié pour l'IA Upsell ?

- ☒ **A** Traitement plus rapide des calculs IA
- ☒ **B** Meilleure gestion de la montée en charge
- ☐ **C** Réduction des coûts d'hébergement
- ☐ **D** Amélioration de la stabilité du service

🔍 Les serveurs GPU sont conçus pour exécuter des calculs massifs en parallèle, ce qui accélère le traitement et permet de gérer plusieurs utilisateurs simultanément sans ralentissement.

### Exercice

Quels sont les avantages et inconvénients d'utiliser une API cloud pour la fonction Upsell ?

- ☒ **A** Avantages : rapidité et simplicité d'intégration
- ☒ **B** Avantages : gestion automatique de la montée en charge
- ☒ **C** Inconvénients : coût par requête
- ☒ **D** Inconvénients : dépendance à un service externe
- ☒ **E** Inconvénients : confidentialité des données

🔍 L'API cloud offre une intégration rapide et une gestion optimisée de la charge, mais engendre des coûts, une dépendance externe et des questions de confidentialité.

### Exercice

Quel est l'inconvénient principal de mettre en place une file d'attente (queue) pour gérer les requêtes IA ?

✓ Augmentation du temps de latence côté utilisateur

Risque de surcharge du serveur

Complexité de développement

Perte de données



La file d'attente garantit que toutes les requêtes sont traitées mais peut augmenter le délai d'attente pour l'utilisateur, ce qui peut nuire à l'expérience, surtout lors du paiement.

### Exercice

Comment calcule-t-on le taux de conversion sur un site e-commerce ?

✓ Nombre de conversions divisé par nombre total de visiteurs multiplié par 100

Nombre de visiteurs divisé par nombre de conversions multiplié par 100

Nombre de commandes divisé par chiffre d'affaires

Chiffre d'affaires divisé par nombre de commandes



Le taux de conversion est le pourcentage de visiteurs qui réalisent l'action attendue, calculé en divisant le nombre de conversions par le nombre total de visiteurs, puis multiplié par 100.

### Exercice

Dans le frontend React, l'utilisateur peut ajouter des produits au panier, demander une suggestion IA et ajouter la suggestion au panier. Vrai ou Faux ?

✓ Vrai

Faux



Le composant React permet de gérer le panier, d'envoyer les articles au backend pour obtenir une suggestion IA, puis d'ajouter cette suggestion au panier si l'utilisateur le souhaite.

### Exercice

Quel est le rôle de l'endpoint /upsell dans le backend FastAPI ?

✓ Recevoir la liste des articles du panier et retourner une suggestion de produit complémentaire

Calculer le panier moyen

Analyser le sentiment des messages clients

Gérer l'authentification des utilisateurs

🔍 L'endpoint /upsell reçoit une requête POST avec les articles du panier, appelle la fonction get\_upsell pour générer une suggestion, puis renvoie cette suggestion au frontend.

### Exercice

Quelles recommandations sont pertinentes pour une version 2 (V2) de la fonction Upsell ?

**A** Utiliser un serveur GPU dédié pour la montée en charge

**B** Tester différentes versions de prompt

**C** Mettre en place une API cloud

**D** Limiter la taille des prompts

**E** Déployer directement sans tests

🔍 Pour une V2, il est conseillé d'améliorer la performance avec un serveur GPU ou API cloud, tester les prompts pour optimiser la pertinence, et limiter la taille des prompts pour éviter les abus. Déployer sans tests est déconseillé.

### Exercice

Associez les concepts à leur définition ou rôle

| Proposer un produit complémentaire pour augmenter le panier moyen | Dépense moyenne par commande | Facilite les tests et améliorations | Optimisé pour calculs IA rapides et gestion de charge |
|-------------------------------------------------------------------|------------------------------|-------------------------------------|-------------------------------------------------------|
| Upsell                                                            | Panier moyen                 | Prompt versionné                    | Serveur GPU dédié                                     |

🔍 Chaque concept est lié à sa définition ou fonction spécifique dans le contexte de la fonction Upsell IA.

### Exercice

Placez dans l'ordre correct les étapes pour intégrer la fonction Upsell dans le backend.

**A** Créer le prompt versionné**B** Centraliser le chargement des prompts**C** Charger le modèle IA**D** Définir la fonction get\_upsell**E** Créer l'endpoint /upsell

L'intégration commence par la création du prompt, sa centralisation, le chargement du modèle, la définition de la fonction métier, puis la création de l'endpoint API.

**Exercice**

Placez dans l'ordre les actions réalisées dans le frontend React pour utiliser la fonction Upsell.

**A** Ajouter des produits au panier**B** Envoyer la liste au backend via POST /upsell**C** Recevoir la suggestion IA**D** Afficher la suggestion**E** Ajouter la suggestion au panier

L'utilisateur construit son panier, demande une suggestion, reçoit et visualise la proposition, puis peut l'ajouter au panier.

**Exercice**

Associez les avantages et inconvénients à l'utilisation d'une API cloud pour l'Upsell

**Rapidité, simplicité, gestion montée en charge****Coût, dépendance, confidentialité**

Avantages

Inconvénients



L'API cloud offre des bénéfices en termes de performance et maintenance, mais présente des limites liées au coût, à la dépendance externe et à la confidentialité.

**Exercice**

Associez les caractéristiques au serveur GPU dédié ou à l'API cloud

**Exécution locale, nécessite gestion montée en charge****Service externe, facturation à l'usage, montée en charge automatique**

Serveur GPU dédié

API cloud



Le serveur GPU est une solution locale avec gestion interne, tandis que l'API cloud est un service externe avec facturation et gestion automatique.

**Exercice**

Placez dans l'ordre les étapes recommandées avant de déployer la fonction Upsell en production.

**A** Tester la fonction en interne**B** Évaluer la montée en charge**C** Choisir entre serveur GPU ou API cloud**D** Mettre en place la gestion des erreurs et files d'attente**E** Déployer progressivement

Le déploiement doit être précédé de tests, d'évaluation de la charge, choix d'infrastructure, gestion des erreurs, puis un déploiement progressif pour limiter les risques.

[exercice p. 31] **Solution n°2**

Le prototype de la fonctionnalité d'upsell a été validé par l'équipe. Il est maintenant temps de préparer la V2 en vue d'un déploiement plus robuste pour l'utilisateur et de présenter ce que fait la fonctionnalité tant côté frontend que backend.

Mais avant cela, commençons par décrire dans les grandes lignes le fonctionnement de notre upsell.

Côté frontend, l'utilisateur peut ajouter des articles à son panier via une sélection de produits (boutons cliquables). Il y a également la possibilité de saisir une information manuellement. Le panier est ensuite affiché dynamiquement, avec la possibilité d'ajouter un produit complémentaire. Les technologies que nous allons utiliser pour le frontend sous React/Router/Vite/Tailwind.

Côté backend, le fonctionnement repose sur une logique modulaire et versionnée. Lorsqu'un appel est reçu, le backend récupère les items du panier. Il charge ensuite un prompt versionné. Le modèle d'IA renvoie une suggestion. Par ailleurs, le modèle utilisé pour ce projet est Mistral en quantification GGUF car il est bien adapté aux contraintes budgétaires de l'entreprise. En somme, les technologies que nous allons utiliser pour le backend sont principalement FastAPI, Pydantic, et LLAMA.

[exercice p. 31] **Solution n°3**

Pour la V2, il est recommandé de migrer vers une API externe comme OpenAI. Une file d'attente (queue) pourrait également être mise en place. Il est également possible de changer de modèle d'IA et/ou de trouver un modèle français open source et léger.